

real-tr-p4-full

An AgentMPI run analysis

Abstract

Four agent ranks translated the eight-section book by the full stencil decomposition — `scatterv`, per-unit terminology extraction, a `UNION allreduce` over the glossary, translation, a `halo_exchange` on a four-node Cartesian line, and `gather` — in 308.44s with no failures, no contract violations and no retries. Speedup over the 576.74s sequential baseline is $1.870\times$ at efficiency 0.467. This is the middle point of the strong-scaling series and the first one at which the topology is a topology rather than a pair, so it is where two claims the campaign makes about the design become testable for the first time, and both survive. The seam-reconciliation phase does real work here: three of the six `revise` calls changed the text, and at the $u_5 | u_6$ boundary both sides edited toward each other coherently, which is the clearest evidence in the series that the halo exchange is not decoration. And a neighbourhood collective is measurably cheaper than a barrier: the exchange blocked its ranks for 64.25 rank-seconds where a global barrier at the same point would have cost 79.91, a 19.6% saving that is 0% at $p = 2$ and 43.3% at $p = 8$. The document’s main negative finding is where this point’s efficiency actually goes. It is not the collectives. It is the `pre-gather` barrier, which absorbed 174.40 rank-seconds — 53.2% of all idle time and 14.1% of the rank-seconds the run had available — because the seam phase hands the two interior ranks two revisions each and the two edge ranks one each, and a rank runs its revisions in sequence. Four ranks is the only configuration in the series with that asymmetry, and it costs this point about a fifth of its wall clock.

1 What this run is

property	value
run	real-tr-p4-full
experiment	translation
campaign label	real-tr
declared world size	4
ranks appearing in the log	10
executors	<code>broker</code> $\times$ 4, <code>worker</code> $\times$ 37
events	227
wall time	308.44s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	2.94×	total busy time over wall time
parallel efficiency	73.4%	achieved parallelism per rank
idle fraction	26.6%	rank-seconds paid for and unused
peak ranks busy	4	of 4 in the world
serial fraction of active time	7.7%	time with exactly one rank busy
load imbalance	1.22×	slowest rank’s busy time over the mean
coordination share	16.2%	rank-seconds blocked in collectives, of those available
coordination in flight	34.8%	wall clock with any rank inside a collective
rank reattachments	31	fresh agent processes that took over a rank role
agent calls	22	invocations that reached a model
agent latency p50 / p95	48.52 / 66.45 s	per invocation
messages	24	21 eager, 3 rendezvous
tokens sent	15,149	2,170 deferred under rendezvous
tokens in / out	39,380 / 7,546	prompt and completion
cost	\$0.2313	0.0307 \$/kTok out

3 What the analysis notices

- **[warning]** `halo_exchange/?` reports 0 messages but the fabric logged 6: the collective’s own accounting disagrees with the traffic it produced.
- **[warning]** Ranks 4, 5, 6, 7, 8, 14 appear in the log without participating; the declared world size is 4. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- **[warning]** 1 collective(s) completed but recorded no duration (`halo_exchange`), so they contribute nothing to the coordination figures even though every participant blocked in them. The reported coordination share is short by exactly their blocking; it can be recovered from the event timestamps.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -13.66$ s), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.624$. Any latency prediction from this run’s calibration is a median, not a model.
- **[ok]** 31 rank reattachment(s) occurred, up to incarnation 8: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.



Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

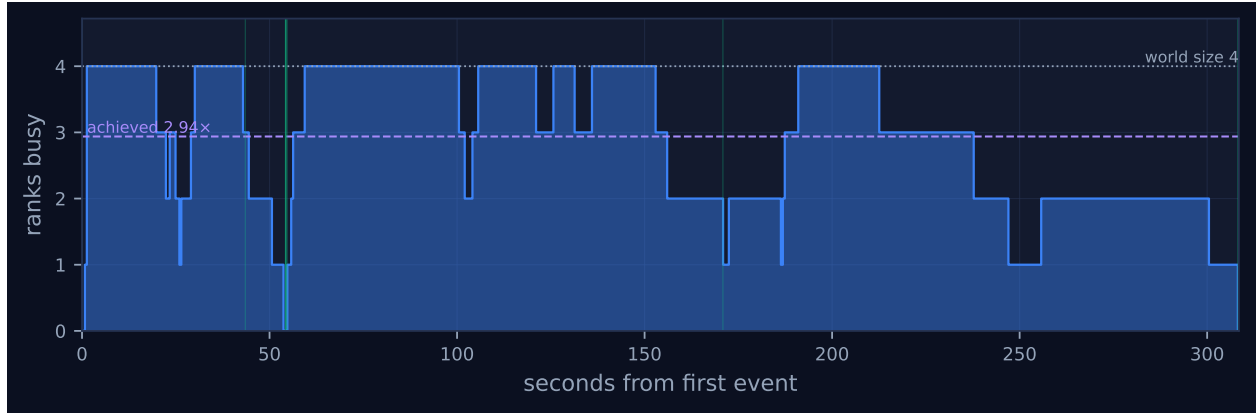


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

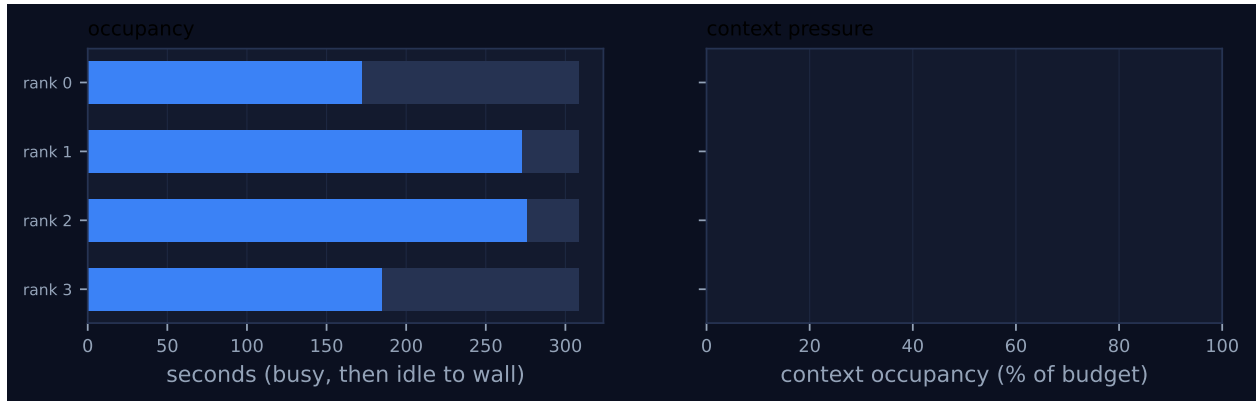


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

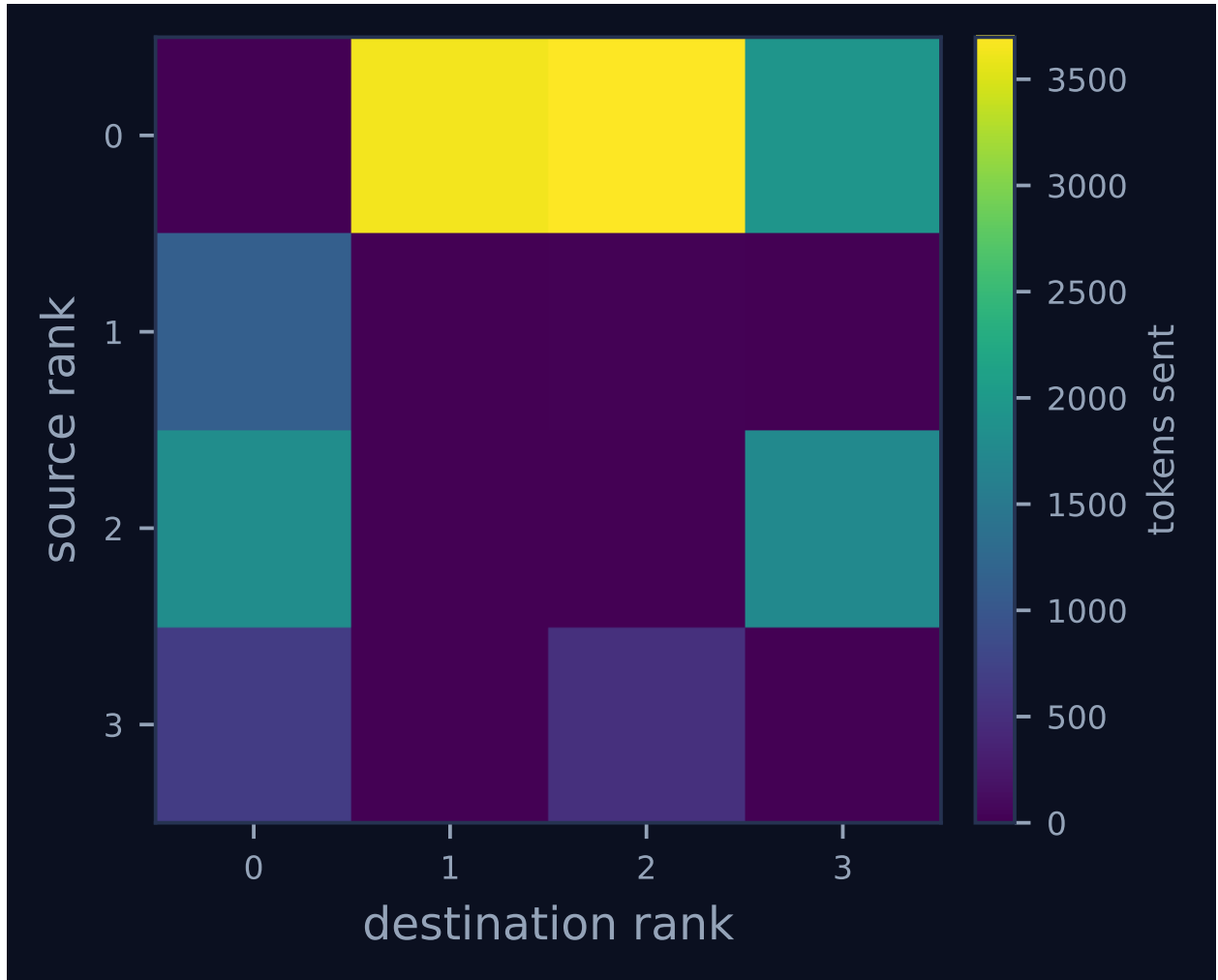


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

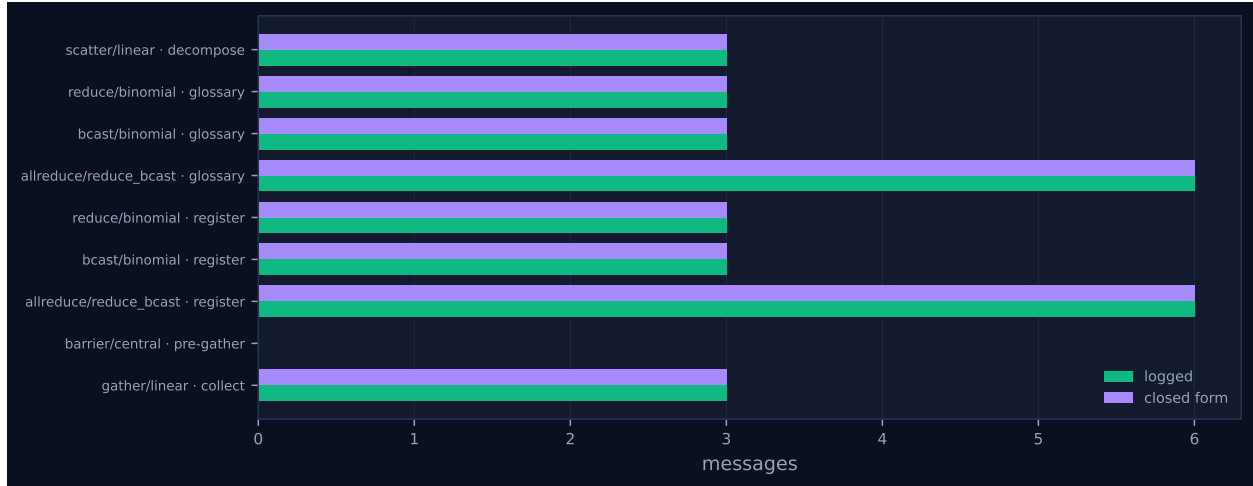


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	172.38	55.9	5	5	8	8	9,263	0.0	finalized
1	272.93	88.5	6	6	5	5	1,146	0.0	finalized
2	275.89	89.5	6	6	7	7	3,571	0.0	finalized
3	185.00	60.0	5	5	4	4	1,169	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	linear	decompose	4	4	1	3	3	3	5,792	0.01	0.01	✓
reduce	binomial	glossary	4	4	2	2	3	3	1,677	9.71	10.71	✓
bcast	binomial	glossary	4	4	2	2	3	3	4,734	10.99	0.28	✓
allreduce	reduce_bcast	glossary	4	4	4	4	6	6	0	10.99	0.28	✓
reduce	binomial	register	4	4	2	2	3	3	138	0.36	0.34	✓
bcast	binomial	register	4	4	2	2	3	3	408	0.43	0.09	✓
allreduce	reduce_bcast	register	4	4	4	4	6	6	0	0.43	0.09	✓
halo_exchange	?	seams	4	4	0	—	6	—	0	0.00	15.82	—
barrier	central	pre-gather	4	4	0	2	0	0	0	95.99	0.19	✓
gather	linear	collect	4	4	1	3	3	3	2,170	0.05	0.19	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 splits cleanly into two halves at $t \approx 186$ s, and they look nothing alike. Before that the four lanes are dense and roughly in step: all four ranks are busy for 111.0 of the first 187 seconds, and Figure 2 sits on its ceiling of four for most of the terminology and translation phases. After it the picture collapses — rank 0's lane ends at 212.4s and rank 3's at 237.5s, while ranks 1 and 2 run bars until 308.0 and 300.3. The last 95.6 seconds of a 308-second run, 31% of it, are two ranks and then one rank working while the other two hold an open barrier. Everything interesting about this point is in that asymmetry, and the rest of this section builds up to it.

Decompose. At $t = 0.009$ rank 0 sends three eager messages of 1,881, 1,984 and 1,927 tokens and the `scatterv` is over in 6 ms. That all three are eager is worth one sentence, because it is a property of p : the ranks publish an `eager_limit` of 2,048 tokens, a linear scatter's hop carries $1/p$ of the book, and at $p = 4$ that is just under the limit where at $p = 2$ the single 3,911-token hop was just over it and went rendezvous. The only rendezvous messages in this run are the three `gather` sends, which the harness forces with `mode=Mode.RENDEZVOUS`, `admit=False`.

Terminology. Each rank runs two `terms` calls back to back. The first four complete between 19.99 and 25.77s and the second four between 43.13 and 53.68s, and Figure 2 shows the notch between the two rounds where ranks finish their first call at different times and the broker takes a few seconds to hand each of them a second.

Glossary. This is where $p = 4$ first exercises a tree. `algorithms.reduce` chooses "flat" if $p \leq 3$ else "binomial", so the glossary reduce is a two-round binomial fan-in: rank 3 sends its 449 tokens to rank 2, rank 1 sends its 400 to rank 0, rank 2 folds and forwards 888 to rank 0. The costs are pure arrival skew. Rank 3 finished its term sheets at 43.53s and sent immediately; rank 2 did not finish until 54.04 and found rank 3's message with a logged queue residency of 10.503s; rank 0 blocked from 44.53 to 54.24, which is the 9.71s the collectives table reports. The broadcast back is the mirror image and adds 10.99s, almost all of it rank 3 waiting at the bottom of the tree. Once the last term sheet existed, the whole reduce-and-broadcast took 0.48s of wall clock and 6 messages, exactly as Figure 5 predicts for $2\lceil \log_2 4 \rceil$ rounds. Every rank recorded the same 208-term glossary and the same broadcast digest 630de966c676.

Translate. Two calls per rank again, and the spread widens: rank 0 finishes at 153.1s, rank 3 at 156.2, rank 1 at 170.7, rank 2 at 186.7. Rank 2's `translate:u4` is the longest agent call in the run at 77.03s, and rank 2 is the rank holding unit 4, which has 43 of the book's 149 paragraphs.

Seams. Each rank calls `cart_create` the instant its last translation returns, so the four ranks enter the exchange 33.5s apart. Six messages move 135 tokens, which is $2(p - 1)$ for a non-periodic line. The traffic is strikingly asymmetric — the right-going messages carry the last paragraph of a rank's last unit and are 43, 26 and 41 tokens, while the left-going messages carry the first paragraph of a rank's first unit and are 5, 9 and 11 — because in this book a section tends to open on a line of dialogue and close on a paragraph of narration. The blocking is 17.74s for rank 0, 15.96 for rank 1, 0.04 for rank 2 and 30.50 for rank 3, 64.25 rank-seconds in total. Rank 2, the straggler, waited for nobody.

Revise, and the barrier. Rank 0 issues one revision and rank 3 one; ranks 1 and 2 issue two each and run them in sequence. Rank 0 finishes at 212.4s and enters the barrier; rank 3 finishes at 237.5 and enters; rank 2 finishes its second at 300.3; rank 1 finishes its second at 308.0 and releases everyone. The barrier's reported wall times are 95.99, 70.72, 7.69 and 0.0001s — 174.40 rank-seconds, which is more than twice the halo exchange and more than five times the entire

glossary `allreduce`. The `gather` that follows is three rendezvous messages of 2,170 tokens and completes in 51 ms.

Partitioning every rank-second by the interval it falls in — boundaries at `broker.enqueue`, `broker.claim`, `broker.complete`, `agent.call`, `msg.recv` and `coll.*` — gives an idle budget that closes:

where the idle rank-seconds went	rank-s	share
pre-gather barrier	174.404	53.2%
halo_exchange	64.246	19.6%
broker dispatch, <code>enqueue</code> to <code>claim</code>	57.928	17.7%
glossary broadcast	15.526	4.7%
glossary reduce	10.398	3.2%
completion polling	4.670	1.4%
scatterv , gather , host-side code, post-finalisation	0.402	0.1%
total idle, of 1,233.774 rank-s available	327.574	100.0%

The 906.201 rank-seconds not in this table are the model working, and they are the 73.4% parallel efficiency in the headline. Two rows deserve emphasis. The barrier alone is larger than every message-passing collective in the run put together. And 57.9 rank-seconds — more than the halo exchange — are the broker publishing a task and waiting for a worker process to take it, 22 times, at a mean of 2.633 s per call.

Figure 4 shows the tree and the ring superimposed. Rank 0 is a source of 9,263 tokens because it scatters; rank 2 is the interior node of the binomial broadcast and relays 1,578 tokens to rank 3 on top of its own traffic, which is why it sends 3,571 where ranks 1 and 3 send about 1,150. The two nine- and twenty-six-token cells $2 \rightarrow 1$ and $1 \rightarrow 2$ are the halo, and they are the only cells in the matrix that exist because of the topology rather than the tree. Figure 3 puts ranks 1 and 2 at 88.5% and 89.5% occupancy against ranks 0 and 3 at 55.9% and 60.0%; its right-hand panel is empty, because every receive in this harness is non-admitting and context occupancy is 0.0% of a 120,000-token budget at all four ranks.

The viewer screenshot shows what Figure 1 filters out. The dashboard draws ten lanes: the four working ranks, and then ranks 4, 5, 6, 7, 8 and 14, each carrying two grey lifecycle ticks and no work at all, in a world declared to be of size 4. The plot is the better picture of the computation and the screenshot the better picture of the job.

7 Interpretation

What is true by construction

The phase structure is the harness’s and the trace matches it instruction for instruction: 4 `coll.scatter`, 8 `coll.reduce`, 8 `coll.bcast`, 8 `coll.allreduce`, 4 `topo.cart_create`, 4 `coll.halo_exchange`, 4 `coll.barrier`, 4 `coll.gather`, and 22 `agent.call`. The 22 calls are eight **terms**, eight **translate** and six **revise**, and the six follow from the decomposition rather than from any choice. With two units per rank, `build_harness` hands neighbour context only to a rank’s first unit (from the left) and its last unit (from the right), so an interior rank revises both of its units and an edge rank revises one: 1, 2, 2, 1.

Nine of the ten collective invocations sent exactly the number of messages their closed form predicts, which Figure 5 shows as matched pairs; the tenth is `halo_exchange`, discussed below.

One property is not by construction and matters for reading the series: $p = 4$ is the first point that takes the `binomial` code path. The selector in `algorithms.reduce` and `algorithms.bcast` is `"flat" if p <= 3 else "binomial"`, so the glossary at $p = 2$ was a one-round flat exchange and here it is a two-round tree. The scaling curve is not algorithmically homogeneous across its four points, and the switch happens between the second and third of them.

Where the efficiency went: a barrier paying for an unbalanced seam phase

This is the finding the run makes best. Efficiency at this point is 0.467, down from 0.703 at $p = 2$, and the idle budget above says the loss is not in the message-passing collectives. The glossary `allreduce` costs 25.92 rank-seconds, 2.1% of what the run had available. The `scatterv` and `gather` together cost 86 ms. The barrier costs 174.40, and the barrier is not a coordination cost in any interesting sense — it is the shape of the work arriving at it.

The shape is this. Model work per rank is 172.38, 272.93, 275.90 and 185.00 seconds, an imbalance of 1.218. Splitting that by phase separates two different causes:

rank	units	words	paragraphs	terms+translate (s)	revisions	revise (s)
0	u_0, u_1	1,324	26	132.49	1	39.90
1	u_2, u_3	1,238	34	156.30	2	116.62
2	u_4, u_5	1,241	69	171.14	2	104.76
3	u_6, u_7	1,306	20	134.89	1	50.11

Through terminology and translation the imbalance is 1.151 and it tracks paragraph count: rank 2 holds 69 of the book’s 149 paragraphs and is the slowest, rank 3 holds 20 and is nearly the fastest, and word count — which is what `split_units` balances — points the wrong way, since rank 0 has the most words and the second-lowest time. The revision phase then doubles the spread, and it does so for a reason that has nothing to do with unit size: ranks 1 and 2 have two revisions and ranks 0 and 3 have one, and a rank executes its `revise` calls in a Python `for` loop over `mine`, one after another. Rank 1 spends 116.62 seconds on revisions where rank 0 spends 39.90.

The consequence is the 174.40 rank-second barrier, and the counterfactual is easy to state. Revision work totals 311.39 rank-seconds; spread evenly over four ranks that is 77.85 each, and the phase’s critical path would be about 78 seconds plus dispatch instead of the 116.6 rank 1 actually took. The run would have finished near 270 s rather than 308 s, for a speedup of about 2.14 instead of 1.870. Roughly a fifth of this point’s shortfall against $p = 2$ ’s efficiency is one scheduling decision inside the harness.

Four ranks is the only configuration in the series where this bites. At $p = 2$ each rank has one boundary and issues one revision; at $p = 8$ each rank holds a single unit and every rank issues exactly one; only at $p = 4$ (and at any other p with more than one unit per rank and more than two ranks) does the interior/edge distinction produce a 1, 2, 2, 1 fan-out. That is a property of the harness, not of AgentMPI, and it is worth reporting as such: the seam phase’s work is distributed by adjacency, and adjacency is not uniform on a line.

The neighbourhood collective earns its keep, and this is where it starts to

The design argument for a `halo_exchange` over a barrier is that a rank should be released when its neighbours arrive rather than when the slowest rank in the world arrives. At $p = 2$ that argument cannot be made, because a rank’s only neighbour is the whole rest of the world. At $p = 4$ it can be made and measured.

The four ranks entered the exchange at 153.14, 156.19, 170.74 and 186.66 s. A global barrier at that point would have held each of them until 186.66 and cost 79.91 rank-seconds. The exchange cost 64.25, so the topology saved 15.66 rank-seconds, 19.6% of the barrier’s cost. The saving is entirely rank 0’s: its only binding neighbour is rank 1, which arrived at 170.74, so it left the exchange 15.8 s before the straggler arrived. Ranks 1 and 3 both wait on rank 2 either way, and rank 2 waits on nobody.

Running the same comparison across the series gives -0.5% at $p = 2$, 19.6% at $p = 4$ and 43.3% at $p = 8$ (34.72 rank-seconds against a 61.27 barrier). The negative value at $p = 2$ is real and not a rounding artefact: a two-rank halo costs marginally *more* than a barrier because it exchanges two messages to achieve the same rendezvous. So the benefit of the neighbourhood collective is zero at the bottom of the curve, first becomes positive here, and is worth almost half the barrier at $p = 8$. This run is the point at which the design choice starts paying, and it is the smallest configuration in which the claim can be tested at all.

The seam mechanism did something here, and the reporting still does not show it

Three of the six revisions altered the text. Pulling the `translate:un` and `revise:un` results out of `runs/real-tr-p4-full/blobs` and diffing the `translation` field gives 22 characters moved out of 6,959: `revise:u4` moved 4, `revise:u5` moved 15, `revise:u6` moved 3, and `revise:u1`, `revise:u2` and `revise:u3` returned their input byte for byte.

The three edits are not noise, and one pair of them is the best evidence in the campaign that the exchange does what it was designed to do. All three fall on the last or first paragraph of a section, which is where the mechanism is aimed. At the $u_5 | u_6$ boundary both sides edited, and they edited toward each other. Rank 2 rewrote unit 5’s closing clause from “laughed heartily for a while” to “laughed on and on for several minutes”; rank 3 rewrote unit 6’s opening verb of speech and its connective so that the choking laughter it describes reads as continuing rather than beginning. Each had seen only the other’s pre-revision text — the two calls ran concurrently, rank 2’s from 190.9 to 246.8 s and rank 3’s from 187.4 to 237.5 — and their own recorded rationales say so explicitly: rank 3’s first `changes` entry is that the new wording “directly continues the laughter at the end of the previous section, so this section does not read like the opening of a new chapter”. At the $u_3 | u_4$ boundary rank 2 inserted a four-character connective into unit 4’s opening so that the speech picks up the gesture that closed unit 3. The remaining seam, $u_1 | u_2$, drew no edit from either side. Two of three reconciled seams were touched.

The `changes` field remains untrustworthy. The harness records `n_boundary_changes` of 3, 8, 4 and 3 for the four ranks, eighteen in total, against three revisions that actually moved a character. Reading the lists shows why: the entries are descriptions of verification — “checked the glossary terms shared with the preceding section and made no further alteration” — because the `Revision` contract asks for “a short list of the boundary or terminology fixes you made” and the agents answered with the items they inspected. `n_boundary_changes` therefore counts self-reported inspections, and nothing in the pipeline diffs the artefacts. Across the series the reported and real counts are 7 against 0 at

$p = 2$, 18 against 3 here, and 30 against 8 at $p = 8$.

What the phase costs is 311.39 rank-seconds of model work plus 64.25 of halo blocking, 375.64 in total, 30.4% of the rank-seconds available and work the sequential baseline never performs at all. Per reconciled seam that is 125.2 rank-seconds here against 157.1 at $p = 2$ and 55.9 at $p = 8$, and the number of seams it can reach is 1, 3 and 7 of the book’s seven. The four seams interior to a rank’s own allocation — $u_0 | u_1, u_2 | u_3, u_4 | u_5, u_6 | u_7$ here — were produced by separate agent calls with no shared context and are exactly as raw as an inter-rank seam, and the mechanism never looks at them.

None of this is visible in the quality metrics: coverage 1.000, consistency 1.000, `n_divergent_entities` 0, rendering verification 1.000 and paragraph fidelity 1.000 at this point and at every sibling, including `real-tr-p8-nohalo`, which has no seam mechanism at all. The campaign has no instrument that would notice whether the 375.64 rank-seconds bought anything.

`coordination_share` is short by exactly the halo

`metrics.json` reports `coordination_share: 0.1625` and `coordination_span_share: 0.3476`, with `coordination_is_underreported: false`. The first two are too low and the third should be true.

`Analysis.collective_rank_seconds` sums `rank_wall_s`, which is assembled from the `wall_s` field each `coll.*` event carries. The `coll.halo_exchange` events in this trace carry only `{label, left, right}`: no `algorithm`, no `rounds`, no `messages_sent`, no `wall_s`. The metric therefore scores the halo at zero. Recomputing blocking directly from event timestamps gives 264.66 rank-seconds against the reported 200.44, and the shortfall of 64.22 is the halo blocking to within 30 ms. Corrected, coordination share is 21.5% rather than 16.3% and the span share is 45.6% rather than 34.8%. The same correction applied across the series gives 10.0%, 21.5% and 19.0% at $p = 2, 4, 8$ against the reported 4.1%, 16.3% and 16.4% — so the corrected figure peaks here rather than rising monotonically, which is the opposite of the shape the uncorrected numbers suggest.

`coordination_is_underreported` exists to warn a reader that the blocking figure is incomplete, but its implementation is `bool(self.incomplete_collectives)` or `self.ok` is `False`. It catches the $p = 16$ failure mode, where ranks die inside a collective and never record it. It cannot catch this one, where the collective completed and recorded itself but omitted its duration. I judge this a real defect in two parts: the instrumentation gap in `topology.halo_exchange`, which the sibling analysis of `real-tr-p8-full` reports as closed by commit `ee21d79` after these traces were taken, and the underreporting flag, which as written would still read `false` on any run whose blocking is invisible for a reason other than failure. Fixing the first does not fix the second for traces already recorded, and every trace in this campaign predates it.

The generated warning that `halo_exchange/?` reports 0 messages against 6 logged is the visible tip of the same gap and is correctly raised. The fabric logged six messages tagged `_ampi:halo:0:8:R` and `_ampi:halo:0:8:L`, which is $2(p - 1)$ for a non-periodic line, and no closed form is registered for the operation, which is why the row shows – and Figure 5 omits it.

The calibration warning names the wrong coefficient

The findings warn that α and β here are a median fallback rather than a fit, “because the regression returned a negative slope ($\beta = 0.1682\text{s/token}$)”. The conclusion is right and should be heeded —

the published $\alpha = 48.517\text{s}$ and $\beta = 0.071984\text{s/token}$ are robust medians, not a model, and no latency prediction should be drawn from them — but the reason is not what happened.

Refitting the 22 invocations reproduces the regression exactly: $\beta = +0.168202\text{s/token}$, a positive slope of about six output tokens per second, with $\alpha = -13.66\text{s}$ and $R^2 = 0.624$. The guard in `cost.calibrate` is `if beta > 0 and alpha > 0`, so what failed was the intercept. The finding text in `scripts/analyze_run.py` hard-codes the phrase “negative slope” for every `median_fallback` case and then explains it with a queueing story: the broker wait entering the measured latency but not the token count. That story would indeed produce a negative slope, and it is the diagnosis `cost.py`’s docstring records from the semantic-glossary run, but it does not apply here. Broker queueing is not inside `latency_s` in this run: the 2.6s dispatch wait lies between `enqueue` and `claim`, and the reported latency begins at the claim. What the data say is that these calls are generation at a near-constant rate with no measurable fixed cost, so the honest model has $\alpha \approx 0$ and $\beta \approx 1/6$, and least squares over a 210–533 token output range puts the intercept slightly below zero.

I count this a real if small defect, in two parts. The message misnames the rejected coefficient, on this run and on $p = 1$ and $p = 2$. More substantively, the $\alpha > 0$ condition rejects a physically sensible zero-intercept fit, and it does so in a way that inverts the quality ordering of this series: $p = 1$, $p = 2$ and $p = 4$ are rejected with R^2 of 0.697, 0.612 and 0.624, while $p = 8$ is accepted as a least-squares fit with $R^2 = 0.250$. The only published regression in the series is the worst of the four.

Six strays in a world of four

The findings flag ranks 4, 5, 6, 7, 8 and 14 as registering without participating. The fabric’s `ranks` table ends with ten rows for `world_size = 4`, six of them in state `running` with leases renewed to thirty minutes past a job that has already emitted `job.finish`. All six appear twice, at incarnations 1 and 2, in two bursts: within the first four seconds of the run, and again between $t = 242$ and $t = 249\text{s}$.

The campaign’s explanation — a shared worker pool, and a worker registering against the wrong job — fits, and this run pins down the pool’s shape. The stray index set here is $\{4, \dots, 8, 14\}$; `real-tr-p2-full`’s is $\{2, \dots, 8, 14\}$ and `real-tr-p8-full`’s is $\{8, 14\}$. In each case it is exactly the pool’s index set minus the world, and the pool held indices 0...8 and 14. Whether AgentMPI accepting these registrations is a defect is settled in the sibling analysis of `real-tr-p8-full`, which traces it to `RankRuntime.register` performing an unconditional `INSERT` without consulting either the `world_size` meta key or `comm_members`; I agree and will not repeat the argument. What this run adds is containment evidence at a third world size: `comm_members` holds four rows, the binomial trees addressed only ranks 0–3, the Cartesian topology was built over four coordinates, the barrier’s `absent` list is empty, and none of the six strays sent or received a byte. The blast radius was nil, and the reason it was nil is again that the misdirected indices happened to fall outside the world rather than inside it.

The 31 reattachments the findings report are a different thing and are expected: 25 of them are the four working ranks turning over a fresh worker process per agent call, which is how this executor runs, and 6 are the strays’ second registration. Rank 2 reaches incarnation 8 because it made six agent calls.

What this point contributes to the series

Wall times are 576.74, 410.11, 308.44 and 166.78s at $p = 1, 2, 4, 8$, so this point’s speedup is $576.74/308.44 = 1.870\times$ at efficiency 0.467 and a Karp–Flatt serial fraction of 0.380 — the middle value of a column that reads 0.422, 0.380, 0.188. A Karp–Flatt that falls with p is the signature of a fixed cost amortising, not of a constant serial section, so the paper’s USL fit of $\sigma = 0.220$, $\kappa = 0.0000$ describes none of the three points; its residuals have the sign pattern $-, -, +$ and it over-predicts this point’s speedup by 0.54.

Efficiency factorises exactly as $E(p) = (T_1/W(1)) \cdot (W(1)/W(p)) \cdot E_{\text{par}}(p)$, with $W(p)$ the model rank-seconds and E_{par} the achieved parallel efficiency in the headline. Here that is $1.1131 \times 0.5718 \times 0.7345 = 0.4675$ against the table’s 0.47. The middle term is the work amplification, and it is the largest single contributor: $W(4) = 906.20$ rank-seconds against the baseline’s 518.16, a factor of 1.749. Of that, 1.148 is the sixteen shared calls simply taking longer — terminology 171.42 rank-seconds against 156.76 and translation 423.38 against 361.40, on byte-identical terminology prompts — and the remaining 1.524 is the six extra `revise` calls. Dividing the amplification out of the whole series gives work-normalised speedups of 1.000, 1.820, 3.270 and 6.895 and a USL fit of $\sigma = 0.025$, $\kappa = 0.0000$, $R^2 = 0.989$, against the wall-clock fit’s 0.873. Most of the reported serial fraction is the parallel runs doing more work, not waiting longer.

Against its neighbours, this point is the worst of the three parallel configurations by efficiency and the best by information content. It is the first with a real tree, the first with a real neighbourhood, the first at which the halo beats a barrier, the first at which the seam revisions change anything, and the only one whose seam phase is unbalanced. If one point in the series has to be read closely, this is a better candidate than the endpoints.

8 Threats to this reading

The counterfactual barrier and the counterfactual balanced revision phase are arithmetic, not experiments. That the exchange blocked its ranks for 64.25 rank-seconds and that the four ranks entered it 33.5s apart are measurements. That a barrier would have cost 79.91 is a calculation which assumes the rest of the schedule is unchanged, and it is not obviously true: holding rank 0 for a further 15.8s would have delayed its `revise:u1` enqueue, which might have found a worker faster or slower. The same caveat applies with more force to the claim that an evenly split revision phase would have finished the run near 270s. Both are order-of-magnitude statements about where the time is, not predictions.

The seam edits are three edits, and I am reading Chinese I cannot fully evaluate. That `revise:u5` and `revise:u6` changed the text is a measurement, and the specific strings are quoted from the blobs. That the two changes are “coherent” — that they revise toward one continuous action across the seam — is my reading of the two paragraphs, and it is the kind of judgement the trace cannot support. It is also a sample of one seam. The opposite reading, that the three edits are incidental rewording and the three no-ops are the honest answer, is not excluded by anything here.

Attributing time to the interval that ends it is well defined but is not attributing cause. “Rank 0 was blocked in the barrier for 95.99s” is a measurement; “rank 0 was blocked because rank 1 had two revisions to run” is an inference, well supported by rank 1’s `revise:u3` completing 0.15s before the barrier released, but an inference. The 57.9 rank-seconds of broker dispatch in the same table are not protocol at all and should not be read as coordination cost, and the 15.5 rank-seconds

attributed to the glossary broadcast are mostly rank 3 waiting for term sheets that had not been written yet.

This is a broker-mediated run and the log cannot see inside a worker. `metrics.json` reports `executors: {broker: 4, worker: 37}`, and the outputs are real Chinese prose with a target-script ratio of 0.857, so this is not a surrogate executor. But all 22 recorded latencies lie within 30 ms of a multiple of 0.5 s, which is what a scripted executor would also produce; I read it as the broker’s `poll_s = 0.5` completion poll because that is its value in the source and no observed observation lag exceeds it.

Two of eight output sections are byte-identical to two other runs’, the pool was shared, and the usual explanation does not hold up. The campaign ran in descending order of p — $p = 8$ at 06:54, this run at 07:03, $p = 2$ at 07:10, $p = 1$ at 07:19 — against one persistent worker pool. Sections 0 and 1 of this run are byte-identical to `real-tr-p2-full`’s and `real-tr-p1-full`’s, and section 0 is also identical to `real-tr-p8-full`’s, whose own unrevised translation of it differs and which reached the shared text only after `revise:u0` moved seven characters. The campaign’s usual verdict on such coincidences is convergence: a binding 208-term glossary, a pinned paragraph count and an agreed register leave a near-deterministic decoder little room. That is testable here, and it fails. `prompt_terms` does not take the glossary, so a `terms:un` prompt is a pure function of the unit and all four runs issue byte-identical terms prompts — 8 of 8 digests match. If the executor were near-deterministic, all 32 results would agree unit by unit; only 6 of the 48 cross-run pairs do. `terms:u4` carries prompt digest `ea002f41` everywhere and returns 36, 38, 38 and 34 terms in 913, 956, 956 and 872 bytes at $p = 1, 2, 4, 8$. Identical input does not give identical output here. What the identities track is placement: over the four runs’ 48 unit-pairs of final output, all 11 pairs in which the unit sat on rank 0 in both runs are byte-identical and none of the other 37 are. That is what reuse confined to one pool slot looks like. It is not proof — a contiguous split puts the lowest-numbered units on rank 0 in every configuration, so “same rank” and “early unit” cannot be separated within this series, and the shared answers were generated rather than served, since this run’s `translate:u0` reports 46.02 s for its 403 output tokens. My reading is dependence between runs rather than convergence. It leaves the timings, message counts and collective structure in this document untouched, but it means the quality column of the scaling table is not four independent draws.

This point uses a code path $p = 2$ does not. Reduce and broadcast run `binomial` here and `flat` at $p = 2$, because the selector is `p <= 3`. Nothing in this document depends on the distinction — at $p = 4$ the two would send the same three messages in two rounds versus one — but any statement of the form “the glossary allreduce costs X as a function of p ” drawn across all four points is fitting across an implementation change.